ANALYSIS OF OUTPUTS

Portfolio Optimization using Branch and Bound and OpenMP

Design and Analysis of Algorithms (23PCIT501L) · Group 01 (B4-Batch) · Cummins College of Engineering for Women, Pune

1. Outputs Compared

Four programs were run on the same problem (20 stocks, 5% units, 20% cap, daily risk limit 0.016). Table 1 lists what each output reports.

Approach	Time (s)	Threads	Work reported by the program
Sequential exhaustive	5577.673	1	35,561,166,195 portfolios evaluated (PASS)
OpenMP exhaustive	395.482 (best)	16	35,561,166,195 portfolios evaluated (PASS) at every thread count
Sequential B&B	12.8467	1	24,265,951 nodes visited; 5,015,033 complete portfolios tested; 35,556,151,162 pruned
OpenMP B&B	2.6059 (best)	16	Node counts are not printed; risk, total and allocation checks all PASS

Table 1: *Outputs of the four approaches*

2. Correctness

- All 13 runs (sequential and parallel, both algorithms, every thread count) return the same portfolio: expected return 0.00146236 per day and risk 0.01598461, with nine stocks held (TPL 20%, MNST 20%, DECK 15%, NVDA, TSCO and ODFL 10% each, AZO, AAPL and IDXX 5% each).
- The exhaustive search enumerated every feasible portfolio, so it fixes the true optimum. Branch and Bound reaching the same value, after discarding 99.9859% of candidates, shows that its bound never removed the optimum.
- The risk constraint is active: $0.016 - 0.01598461 = 1.5 \times 10^{-5}$. Branch and Bound only checks risk on complete portfolios, so the whole 5,015,033-portfolio tested set was needed to find the best feasible one.

3. Effect of Pruning (algorithm)

Branch and Bound tested 5,015,033 complete portfolios against 35,561,166,195 for the exhaustive search, a reduction of 7091×. The sequential time fell from 5577.67 s to 12.8467 s, a smaller factor of 434×. The cost per unit of work differs: the exhaustive search spent about 157 ns per evaluated portfolio, while Branch and Bound spent about 529 ns per visited node (it also pays for the bound lookup and for internal nodes). Pruning is therefore the dominant gain, far larger than anything achieved by threads.

4. Effect of Parallelism

Table 2 gives speedup and efficiency relative to each program's own 1-thread run, which keeps the comparison within one machine and one code version.

Threads	Exhaustive speedup	Exhaustive efficiency	B&B speedup	B&B efficiency
1	1.00	100.0%	1.00	100.0%
4	1.56	39.1%	1.04	26.1%
6	2.09	34.8%	–	–
8	2.22	27.8%	1.67	20.9%
10	2.30	23.0%	2.00	20.0%
16	2.86	17.9%	2.74	17.1%

Table 2: *Relative speedup and efficiency*

Both programs scale in the same way: speedup of 2.86 (exhaustive) and 2.74 (B&B) at 16 threads, efficiency below 20%. The implied serial fraction (Amdahl) is about 0.31 and 0.32, which is far larger than the real serial part of the code (reading is excluded and the merge is a 25-entry critical section). The loss therefore comes from uneven work, not from synchronization.

5. Why Scaling Is Limited

5.1 Exhaustive search: work per unit is highly uneven

The exhaustive search does the same amount of work for a unit as the number of feasible portfolios beneath it, which can be counted exactly. Unit W0 (first two stocks at 0%) holds 19.7% of all portfolios, W1 holds 12.1%, W5 12.1%, while units such as W24 (20% and 20%) hold under 0.001%. Because W0 alone is 19.7% of the work, no thread count can give more than 5.09× speedup with this decomposition. Table 3 predicts the speedup under static scheduling from these counts and compares it with the measurement.

Threads	Predicted speedup (work counts)	Observed speedup	Observed / predicted
4	1.55	1.56	101%
6	2.21	2.09	95%
8	2.33	2.22	96%
10	2.57	2.30	90%
16	3.15	2.86	91%

Table 3: *Exhaustive search: predicted and observed speedup under static scheduling*

The observed values follow the prediction closely (95–101% of it up to 8 threads and 90–91% at 10 and 16 threads), so load imbalance from static scheduling over 25 uneven units accounts for almost all of the lost efficiency. The small remaining gap at 10 and 16 threads is consistent with hardware limits on the test machine.

5.2 Branch and Bound: heaviest thread bound

Each B&B work unit starts with its own empty best, so the work in a unit does not depend on the thread count. The run time is then set by the most loaded thread, and its share of the 1-thread work is at least T_p / T_1 (Table 4).

Threads	T_p (s)	T_p / T_1	Meaning
4	6.8298	95.8%	Most loaded thread holds ≥ 96% of the work
8	4.2712	59.9%	Most loaded thread holds ≥ 60% of the work
10	3.5674	50.0%	Most loaded thread holds ≥ 50% of the work
16	2.6059	36.5%	Most loaded thread holds ≥ 37% of the work

Table 4: *Branch and Bound: lower bound on the heaviest thread's share*

With 4 threads the heaviest thread still holds about 96% of all work, so the work is concentrated in a few units, and even 16 threads (one or two units each) leave one thread with at least 36%. This explains the near-zero gain from 1 to 4 threads (1.04×). The granularity limits of Table 7 in the report (3.57 at 4 threads, 12.5 at 16) are never approached, which shows that cost skew, not the number of units, is the binding factor.

6. Parallel vs Sequential Baseline

The 1-thread parallel exhaustive run takes 1129.86 s against 5577.67 s for the sequential program (4.94× faster), and the 1-thread parallel B&B takes 7.1307 s against 12.8467 s. Speedups over the sequential programs are therefore larger than relative speedups (exhaustive 14.10× and B&B 4.93× at 16 threads) and include this code-level difference as well as parallelism.

7. Conclusions

- All approaches give the same optimal portfolio; correctness does not depend on the algorithm or thread count.
- Pruning gives the largest improvement (434× sequentially); threads give at most 2.86× (exhaustive) and 2.74× (B&B) at 16 threads.
- Poor efficiency is explained by uneven work among the 25 top-level units combined with static scheduling. For the exhaustive search this is confirmed quantitatively (Table 3).
- With the current split, the exhaustive search cannot exceed 5.09× at any thread count. A deeper split (more, smaller units) with dynamic scheduling is the change that addresses the measured limitation.

Validity of the comparison

The OpenMP exhaustive runs were made on the HP Victus laptop (Ubuntu) and the OpenMP B&B runs on the MacBook Air (Apple M4, 10 cores); thread-scaling conclusions compare runs within one program and machine. Absolute times across programs should be read with that in mind. Machine used for the sequential programs: The sequential Branch and Bound program was run on the Lenovo ThinkPad, while the sequential exhaustive search was run on the Acer Nitro ANV15-41 laptop. Each execution time is based on a single run.